The Journal of Open Source Software

TDAvec: Computing Vector Summaries of Persistence Diagrams for Topological Data Analysis in R and Python

Aleksei Luchinsky[1¶] **and Umar Islambekov**[1*]

1 Bowling Green State University, USA ¶ Corresponding author * These authors contributed equally.

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor:

Submitted: 18 September 2024
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

DRAFT

Summary

The theory of *persistent homology* is one of the popular tools in *topological data analysis* (TDA) to analyze data with underlying shape structure (Carlsson, 2009; Chazal & Michel, 2021; Edelsbrunner & Harer, 2010). In this context, a single data observation could be a collection of points lying in a metric space, an image, a graph or a time series. The basic idea behind persistent homology is to build a nested sequence (or *filtration*) of *simplicial complexes* (indexed by a scale parameter) on top of data points and keep a record of the appearance and disappearance of various topological features at different scale values. Here, these topological features are "holes" of different dimensions – connected components, loops, voids, and their higher-dimensional versions whose emergence and subsequent disappearance are tracked using a concept of homology from algebraic topology. From a geometric point of view, simplicial complexes consist of vertices, edges, triangles, tetrahedra etc., glued together and serve as a means for recovering (at least partially) the underlying shape information which is lost during sampling (Nanda & Sazdanovic, 2013).

A topological descriptor outputted by the persistent homology encoding the shape of data is called a *persistence diagram* (PD). Mathematically, a k-dimensional PD is a multi-set of points $D = \{(b_i, d_i)\}_{i=1}^{N}$, where each point (b_i, d_i) corresponds to a topological feature of homological dimension k (0 if a connected component, 1 if a loop, 2 if a void, etc) with the x-coordinate representing the scale at which this feature is born (or created), and the y-coordinate representing the scale at which it dies (or disappears). In practice, one is usually interested in applying a machine learning method to PDs to make further inferences from data. However, the fact that PDs do not form a Hilbert space, which is a feature (or an input) space for a wide class of machine learning methods, limits their direct of use in applications. To overcome this challenge, kernel methods and vectorization techniques are commonly used (Chung & Lawson, 2022). The kernel approach involves defining a notion of similarity between pairs of PDs, whereas the vectorization methods aim to transform PDs into finite-dimensional feature vectors that can be used as input for many standard machine learning models. Such vector summaries of PDs are computed in two steps: first one constructs a respective summary function from a given PD and then vectorizes it using either one or two dimensional grid of scale values. In recent years, the kernel and vectorization approaches have proven successful and gained prominence in the applied TDA literature (see Hensel et al. (2021) for a survey of applications of TDA in machine learning).

R tools for TDA

The computational tools for TDA in the R environment are provided through various packages[1] such as `TDA` (Fasy et al., 2021), `TDAstats` (R. R. Wadhwa et al., 2018), `kernelTDA` (Padellini &

[1] In this section, we only focus on R packages for TDA that are available on the CRAN repository.

Palini, 2020), TDAmapper (Pearson et al., 2015), TDAkit (You & Yu, 2021), tdaunif (Brunson et al., 2024), TDApplied (Brown & Farivar, 2024) and ripserr (R. Wadhwa et al., 2020) (see Table 1 for an overview of these packages in terms of their scope and areas of focus).

	R libraries								Python			
	TDA	TDApplied	TDAstats	TDAkit	kernelTDA	tdaunif	ripserr	TDAmapper	Giotto-tda	Gudhi	Dionysus 2	scikit-tda
Sampling methods	✓			✓		✓						
Density estimation	✓									✓		
Alpha filtration	✓									✓		✓
Alpha shape filtration	✓											
Vietoris-Rips filtration	✓		✓	✓			✓			✓	✓	✓
User-defined filtration	✓											
Cubical complex							✓		✓	✓		
Wasserstein distance	✓	✓	✓		✓				✓	✓	✓	✓
Plotting persistence diagrams	✓	✓	✓	✓					✓	✓	✓	✓
Statistical methods	✓	✓	✓	✓								
Vectorization methods	✓			✓	✓				✓	✓		✓
Kernel methods		✓		✓	✓							
Supervised learning methods		✓			✓							
Clustering methods	✓	✓		✓					✓	✓		
HD data plots								✓	✓	✓	✓	
Dimension reduction		✓										✓

Table 1: R and Python packages for TDA

The TDA package is the largest R package for TDA. The TDA package offers tools to compute PDs for commonly used types of filtrations such as *Vietoris-Rips*, *Alpha* and *Alpha shape*. It also allows to construct more general sublevel set filtrations and compute the corresponding PDs. Moreover, the TDA package provides implementations to plot PDs and compute *bottleneck and Wasserstein* distances between them. TDAstats offers a variety of tools for conducting statistical inference (such as hypothesis testing) on PDs. Compared to the TDA package, it computes PDs much faster for Vietoris-Rips filtrations based on the Ripser C++ library (Bauer, 2021) and offers more aesthetic visualization of the diagrams using the ggplot2 package (Wickham, 2016). The kernelTDA package contains implementations of popular kernel-based methods for TDA such as *geodesic Gaussian kernel*, *geodesic Laplacian kernel*, *persistence Fisher kernel* and *persistence sliced Wasserstein kernel*. For computing the Wasserstein distance between a pair of PDs, unlike the TDA package, it uses an iterative procedure to reasonably approximate the exact distance which that leads to a considerable reduction in run-time cost. The ripserr package allows a fast computation of PDs for filtrations on Vietoris-Rips and cubical complexes using the Ripser C++ library. The TDApplied and TDAkit packages provides various tools to integrate topological features (PDs or their vector summaries) into machine and statistical learning settings. The tdaunif is a useful package if one needs to sample points from various manifolds such as a klein-bottle, an ellipse or a torus. TDAmapper offers tools to visualize high-dimensional data by constructing the so-called Mapper graphs that preserve its topological structure.

Python tools for TDA

Several Python libraries are available for TDA, such as Giotto-tda (Tauzin et al., 2021), Ripser (Christopher et al., 2018), Gudhi (Rouvreau, 2020), Dionysus 2 (Morozov, 2018), Scikit-tda (Saul & Tralie, 2019) which is a container library for such projects as Ripser, Persim (Saul, 2019), KeplerMapper (Van Veen et al., 2019), etc. Giotto-tda is a powerful

DRAFT

library that integrates with the popular machine learning library `scikit-learn`, offering tools for persistent homology and visualizations of persistence diagrams. `Ripser` focuses on fast computation of Vietoris-Rips complexes, especially for large datasets. `Gudhi` provides a wide range of topological tools for simplicial complexes, persistent homology, and topological signatures. `Dionysus 2` offers fast computation of persistent homology and cohomology, with an emphasis on flexibility and efficiency. `Persim` focuses on tools for working with PDs. It contains implementations of commonly used vectorization and kernel methods for PDs. `KeplerMapper` implements the TDA Mapper algorithm to visualize high-dimensional data. `Scikit-tda` is another package that integrates with `scikit-learn`, simplifying the application of TDA to typical machine learning tasks.

For a more comprehensive list of `Python` libraries for TDA and their functionality, we refer the readers to (*Awesome-TDA*, 2024).

The rest of the paper is organized as follows. In the next section we describe in more details our package and prove its necessity. Session 3 shows some usage examples and the last section is reserved for discussion. In the appendix you will find definitions of all used in our work vectorization summaries.

Statement of need

As it was mentioned in the previous section, vectorization step is extremely important to include TDA in the ML pipeline. Up to now lots of different vectorization methods were proposed (see the Appendix below for short description of some of the existing ones). It turns out that performance accuracy of the ML algorithms depends strongly on the choice of the used vectorization, so it could be very interesting to be able to compare different approaches.

In order to do such a comparison it would be useful to have all considered vectorization methods implemented in one package in the uniform manner. Unfortunately, up to now such a package does not exist. Some mentioned in the Introduction libraries do have implementation of some of the vectorization (see table 2 for quick comparison), but there is no ackage combining all of them together. More over, in R all the code behind the existing vector implementation is written using standard R factions, which may prove slow and inefficient in large-scale computations. In addition, the interfaces of various factions in different packages are not compatible with each other, which makes the comparison even more chalenging.

	R libraries					Python		
	`TDA`	`TDApplied`	`TDAstats`	`TDAkit`	`kernelTDA`	`Giotto-tda`	`Gudhi`	`scikit-tda`
Bar Codes	✓	✓	✓	✓				
Persistence Landscape	✓			✓		✓	✓	✓
Persistence Silhouette	✓			✓		✓	✓	
Persistent Entropy Summary							✓	
Betti Curve						✓	✓	
Euler characteristic curve								
The normalized life curve								
Persistence Surface					✓	✓	✓	✓
Persistence Block								

Table 2: Vectorizaions in different TDA packages

The TDArea R package and its Python counterpart aim to fill these gaps. Its contributions can be summarized as following:

1. It expands the list of implemented vector summaries of PDs by providing vectorization of eight functional summaries found in the TDA literature: *Betti function*, *persistence landscape function*, *persistence silhouette function*, *persistent entropy summary function* (Atienza et al., 2020), *Euler characteristic curve* (Richardson & Werman, 2014), *normalized life curve* (Chung & Lawson, 2022), *persistence surface* (Adams et al., 2017) and *persistence block* (Chan et al., 2022) (see the Appendix for their definitions).

2. A univariate summary function f of a PD is typically vectorized by evaluating it at each point of a superimposed one dimensional grid and arranging the resulting values into a vector:
$$(f(t_1), f(t_2), \dots, f(t_n)) \in \mathbb{R}^n, \tag{1}$$
where $t_1, t_2, \dots, t_n$ form an increasing sequence of scale values. For example, the `landscape()` and `silhouette()` functions of the `TDA` package compute vector summaries of persistence landscapes and silhouettes in this manner. The `TDAvec` package instead employs a different vectorization scheme which involves computing the average values of f between two consecutive scale values t_i and t_{i+1} using integration:
$$\left(\frac{1}{\Delta t_1}\int_{t_1}^{t_2} f(t)dt, \frac{1}{\Delta t_2}\int_{t_2}^{t_3} f(t)dt, \dots, \frac{1}{\Delta t_{n-1}}\int_{t_{n-1}}^{t_n} f(t)dt\right) \in \mathbb{R}^{n-1}, \tag{2}$$
where $\Delta t_i = t_{i+1} - t_i$. Unlike (1), this vectorization method does not miss the behavior of f between neighboring scale points and applies to all univariate summary functions which are easy to integrate, namely persistence silhouette, persistent entropy summary function, Euler characteristic curve, normalized life curve and Betti function.

3. To achieve higher computational efficiency, all code behind the vector summaries of `TDAvec` is written in C++. For example, in `R`, computing the persistence landscape from a PD with the `TDAvec` package is more than 200 times faster than with the `TDA` package.

The `TDAvec` `R` package and a vignette showing its basic usage with examples are available on the CRAN repository[2]. For `Python` examples, we refer the readers to sample notebook presented in (Luchinsky & Islambekov, 2024).

Usage Examples

Let us first describe how the R library TDAvec can be installed and used.

The current version of this library is available on CRAN, so the simplest way to install it is by using the standard R method:

```
> install.packages("TDAvec")
```

Alternatively you can install it from the GitHub repository:

```
> devtools::install_github("ALuchinsky/tdavec")
```

After downloading the library, you can use functions such as computePL, computePS, etc., to calculate the corresponding vectorization summaries of persistence diagrams.

Suppose we have a random set of squeeze factors $e^a \in [0, 1]$, and for each of them, we create a cloud of points located around an ellipse:

$$(x_i, y_i)^a = (r_i \cos\phi_i, e^a r_i \sin\phi_i)$$

Here are R commands to reate the point clouds:

[2] https://cran.r-project.org/web/packages/TDAvec/index.html

```
> epsList <- round(as.vector(read.csv("./epsList.csv", header = FALSE))[[1]], 3)
> clouds <- lapply(epsList, function(e) createEllipse(100, a=1, b=e))
> PDs <- lapply(1:length(clouds), function(i)
        ripsDiag(clouds[[i]], maxdimension = 1, maxscale = 2)$diagram
    )
```

In the figure figure Figure 1 below you can see examples of the created data.

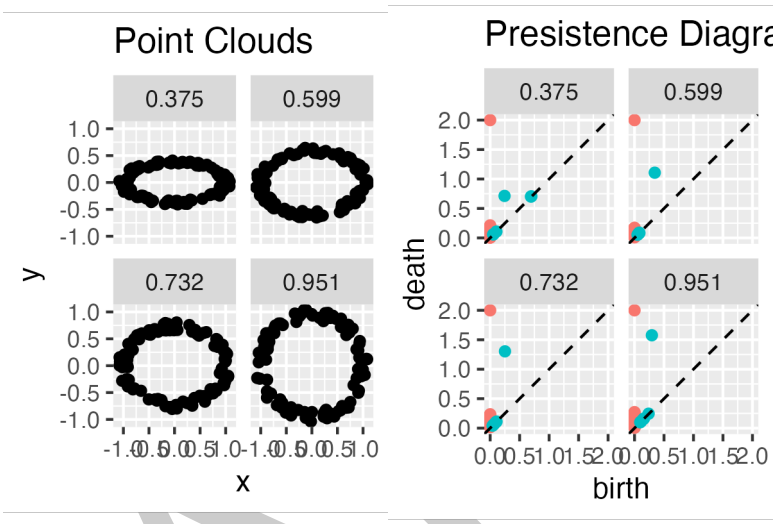

Figure 1: Examples of the point clouds

Created point clouds can be converted into persistence diagrams using such functions as `ripsDiag` from `TDA` package. Each of the for each of the diagrams we can calculate Persistence Landscape summary with the help of `computePL(diagram, homDim, x)` function. In figure Figure 2 you can see the result.

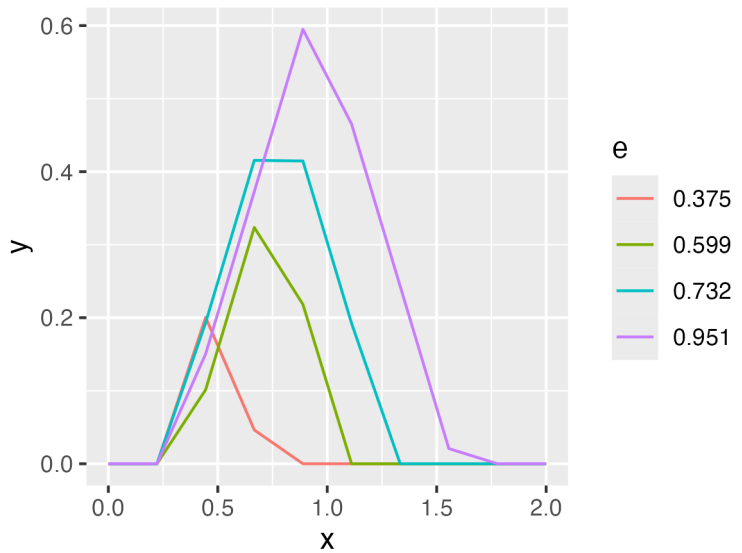

Figure 2: Persistence landscape summaries created from presented above point clouds

DRAFT

The python interface of TDAvec library can be installed from the GitHub repository

```
> pip install git+https://github.com/ALuchinsky/tdavec
```

Note that it was tested on Python 3.11 and uses numpy (vesrion numpy==1.2) and ripser (version 0.6.8) libraries.

You can directly access all available in R interface unctions of the package, like computePL, etc. In addition, python implementation follows the scikit-learn interface, so it uses a class-oriented approach. After installing the library, you can load all the objects with:

```
> from tdavec.TDAvectorizer import TDAvectorizer, createEllipse
```

You can now use an object of the class TDAvectorizer to create persistence diagrams and vectorize them. Here is an example of Python code that performs the same job as the R commands above:

```
> epsList = np.random.uniform(low = 0, high = 1, size = 500)
> clouds = [createEllipse(a=1, b=eps, n=100) for eps in epsList]
> v = TDAvectorizer()
> v.fit(clouds)
```

The last line effectively uses the ripser library to convert data point clouds into a list of persistence diagrams, and the results will be saved in the v.diags property of the vectorizer object. To perform vectorization, we can simply call the fit() method of the same object:

```
> v.setParams({"scale":np.linspace(0, 2, 10)})
> X = v.transform(output="PL", homDim=1)
```

Note that the scale sequence (i.e., grid points for persistence landscape calculation) is specified in the first line. The type of vectorization summary can be specified as a parameter. This approach makes it extremely convenient to easily switch between methods in the analysis and assess accuracy. Here's how we can conduct such an analysis.

First, we define a helper function that vectorizes persistence diagrams using the specified method, builds a linear regression model based on the obtained predictors, and reports accuracies on the training and testing subsets:

```
> def makeSim(method, homDim, vec = v, y=epsList):
>     X =v.transform(output=method, homDim=homDim)
>     Xtrain, Xtest, ytrain, ytest = train_test_split(X, y, train_size=0.8, random_state=
>     model = LinearRegression().fit(Xtrain, ytrain)
>     test_preds = model.predict(Xtest)
>     score = model.score(Xtest, ytest)
>     res = {"method":method, "homDim":homDim, "test_preds":test_preds, "y_test":ytest, "
>     return res
```

Now we can use this function to collect data, construct accuracy tables, and compare true values and model predictions for each vectorization method:

```
> methodList = ["vab", "ps", "nl", "ecc", "fda"]
> df = pd.DataFrame()
> for homDim in [0, 1]:
>     for method in methodList:
>     df = pd.concat([df, pd.DataFrame( makeSim(method, homDim))])
```

Using the created data frame, we can easily compare performance accuracies (see table below). It's also simple to check the accuracy of the models by inspecting true vs. predicted scatter plots (see figure).

DRAFT

The Journal of Open Source Software

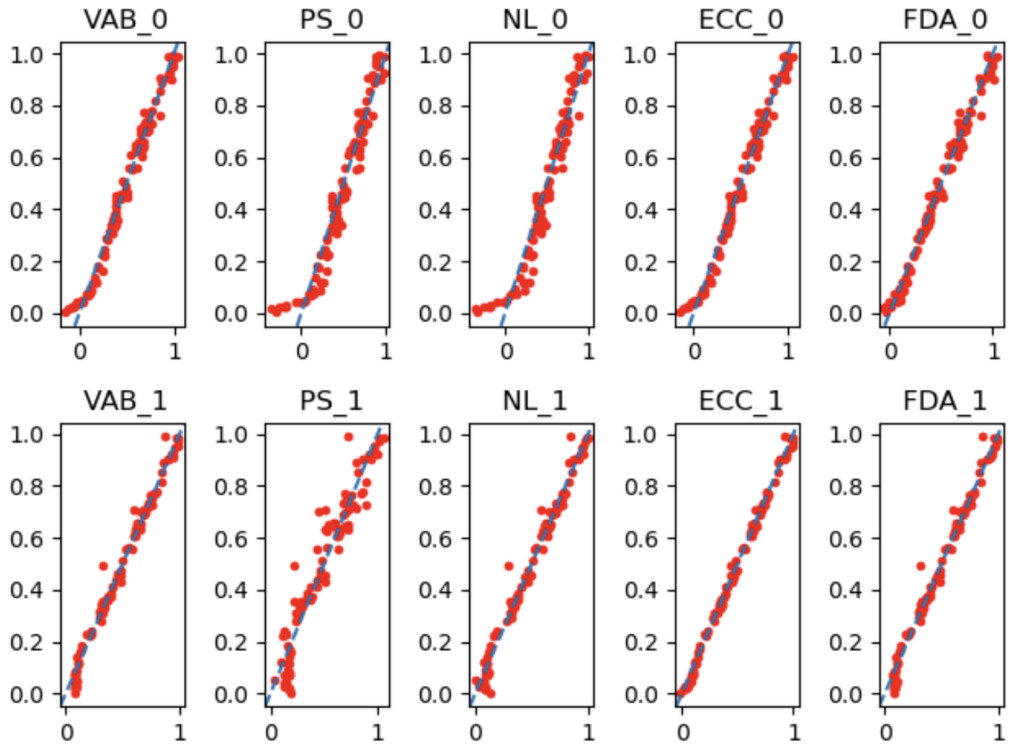

Figure 3: True-Predictions scatter plots for different vectorization methods

Method	dim 0	dim 1
ECC	0.976	0.996
FDA	0.983	0.985
NL	0.8930	0.980
PS	0.903	0.914
VAB	0.976	0.985

DRAFT

Conclusion

Appendix: Definitions of the summary functions in TDAvec

Let $D = \{(b_i, d_i)\}_{i=1}^N$ be a persistence diagram.

1) *The kth order persistence landscape function* of D is defined as

$$\lambda_k(t) = k\text{max}_{1 \le i \le N} \Lambda_i(t), \quad k \in N,$$

where $k$max returns the kth largest value and

$$\Lambda_i(t) = \begin{cases} t - b_i & t \in [b_i, \frac{b_i+d_i}{2}] \\ d_i - t & t \in (\frac{b_i+d_i}{2}, d_i] \\ 0 & \text{otherwise} \end{cases}$$

2) *The pth power persistence silhouette function*:

$$\phi_p(t) = \frac{\sum_{i=1}^N |d_i - b_i|^p \Lambda_i(t)}{\sum_{i=1}^N |d_i - b_i|^p},$$

where

$$\Lambda_i(t) = \begin{cases} t - b_i & t \in [b_i, \frac{b_i+d_i}{2}] \\ d_i - t & t \in (\frac{b_i+d_i}{2}, d_i] \\ 0 & \text{otherwise} \end{cases}$$

3) *The persistent entropy summary function*:

$$S(t) = -\sum_{i=1}^{N} \frac{l_i}{L} \log_2 (\frac{l_i}{L}) \mathbf{1}_{[b_i,d_i]}(t),$$

where $l_i = d_i - b_i$ and $L = \sum_{i=1}^{N} l_i$.

4) *The Betti Curve*:

$$\beta(t) = \sum_{i=1}^{N} w(b_i, d_i) \mathbf{1}_{[b_i,d_i)}(t),$$

where the weight function $w(b, d) \equiv 1$.

5) *The Euler characteristic curve*:

$$\chi(t) = \sum_{k=0}^{d} (-1)^k \beta_k(t),$$

where $\beta_0, \beta_1, \dots, \beta_d$ are the Betti curves corresponding to persistence diagrams $D_0, D_1, \dots, D_d$ of dimensions $0, 1, \dots, d$ respectively, all computed from the same filtration.

6) *The normalized life curve*:

$$sl(t) = \sum_{i=1}^{N} \frac{d_i - b_i}{L} \mathbf{1}_{[b_i,d_i)}(t),$$

where $L = \sum_{i=1}^{N} (d_i - b_i)$.

7) *Persistence Surface*:

$$\rho(x, y) = \sum_{i=1}^{N} f(b_i, p_i) \phi_{(b_i,p_i)}(x, y),$$

where $\phi_{(b_i,p_i)}(x, y)$ is the Gaussian distribution with mean (b_i, p_i) and covariance matrix $\sigma^2 I_{2\times 2}$ and

$$f(b, p) = w(p) = \begin{cases} 0 & p \le 0 \\ p/p_{max} & 0 < p < p_{max} \\ 1 & p \ge p_{max} \end{cases}$$

is the weighting function with p_{max} being the maximum persistence value among all persistence diagrams considered in the experiment.

8) *The persistence block*:

$$f(x, y) = \sum_{i=1}^{N} \mathbf{1}_{E(b_i,p_i)}(x, y),$$

where $E(b_i, p_i) = [b_i - \frac{\lambda_i}{2}, b_i + \frac{\lambda_i}{2}] \times [p_i - \frac{\lambda_i}{2}, p_i + \frac{\lambda_i}{2}]$ and $\lambda_i = 2\tau p_i$ with $\tau \in (0, 1]$.

9) *Functional Data Analysis*:

DRAFT

The drawback of all listed above vectorization methods is that we have to choose some grid points on t axis or on (b, p) plane to get the numeric array of predictors from the vectorized functions. An alternative approach is to select some set of basis functions $\xi_i(t)$ and expand the result of the vectoriztion (for example, Betti Curve) as a combination of them:

$$g(t) = \sum_i I_{[b_i, d_i)]}(t) = \sum_k c_k \xi_k(t)$$

The coefficiets $\{c_k\}$ will be predictors in this approach. It is clear for this type of selection we do not need to select arbitrary the grid points. In addition, it is more robust to small changes of the function $g(t)$ or, alternatively, to birth-death points in the persistence diagram.

References

Adams, H., Emerson, T., Kirby, M., Neville, R., Peterson, C., Shipman, P., Chepushtanova, S., Hanson, E., Motta, F., & Ziegelmeier, L. (2017). Persistence images: A stable vector representation of persistent homology. *The Journal of Machine Learning Research*, *18*(1), 218–252.

Atienza, N., Gonzalez-Diaz, R., & Soriano-Trigueros, M. (2020). On the stability of persistent entropy and new summary functions for topological data analysis. *Pattern Recognition*, *107*, 107509. https://doi.org/10.1016/j.patcog.2020.107509

Awesome-TDA. (2024). https://github.com/FatemehTarashi/awesome-tda

Bauer, U. (2021). Ripser: Efficient computation of Vietoris-Rips persistence barcodes. *J. Appl. Comput. Topol.*, *5*(3), 391–423. https://doi.org/10.1007/s41468-021-00071-5

Brown, S., & Farivar, R. (2024). *TDApplied: Machine learning and inference for topological data analysis*. https://doi.org/10.32614/cran.package.tdapplied

Brunson, J. C., Demkowicz, B., & Choudhary, S. (2024). *Tdaunif: Uniform manifold samplers for topological data analysis*. https://doi.org/10.32614/CRAN.package.tdaunif

Carlsson, G. (2009). Topology and data. *Bulletin of the American Mathematical Society*, *46*(2). https://doi.org/10.1090/S0273-0979-09-01249-X

Chan, K. C., Islambekov, U., Luchinsky, A., & Sanders, R. (2022). A computationally efficient framework for vector representation of persistence diagrams. *Journal of Machine Learning Research*, *23*(268), 1–33.

Chazal, F., & Michel, B. (2021). An introduction to topological data analysis: Fundamental and practical aspects for data scientists. *Frontiers in Artificial Intelligence*, *4*. https://doi.org/10.3389/frai.2021.667963

Christopher, T., Nathaniel, S., & Rann, B. (2018). A lean persistent homology library for python. *Open J*, *925*.

Chung, Y.-M., & Lawson, A. (2022). Persistence curves: A canonical framework for summarizing persistence diagrams. *Advances in Computational Mathematics*, *48*(1), 1–42. https://doi.org/10.1007/s10444-021-09893-4

Edelsbrunner, H., & Harer, J. (2010). *Computational topology: An introduction*. American Mathematical Soc.

Fasy, B. T., Kim, J., Lecci, F., Maria, C., Millman, D. L., & Rouvreau., V. (2021). *TDA: Statistical tools for topological data analysis*. https://doi.org/10.32614/CRAN.package.TDA

Hensel, F., Moor, M., & Rieck, B. (2021). A survey of topological machine learning methods. *Frontiers in Artificial Intelligence*, *4*, 681108. https://doi.org/10.3389/frai.2021.681108

DRAFT

Luchinsky, A., & Islambekov, U. (2024). *Python interface to TDAvec library*. https://github.com/ALuchinsky/tdavect

Morozov, D. (2018). *Dionysus 2 – library for computing persistent homology*. https://github.com/mrzv/dionysus

Nanda, V., & Sazdanovic, R. (2013). Simplicial models and topological inference in biological systems. In *Discrete and topological models in molecular biology* (pp. 109–141). Springer. https://doi.org/10.1007/978-3-642-40193-0_6

Padellini, T., & Palini, F. (2020). *kernelTDA: Statistical learning with kernel for persistence diagrams*. https://CRAN.R-project.org/package=kernelTDA

Pearson, P., Muellner, D., & Singh, G. (2015). *TDAmapper: Topological data analysis using mapper*. https://github.com/paultpearson/TDAmapper

Richardson, E., & Werman, M. (2014). Efficient classification using the euler characteristic. *Pattern Recognition Letters*, *49*, 99–106. https://doi.org/10.1016/j.patrec.2014.07.001

Rouvreau, V. (2020). *GUDHI user and reference manual*. GUDHI Editorial Board.

Saul, N. (2019). *Persim 0.3.7*. https://persim.scikit-tda.org/en/latest/

Saul, N., & Tralie, C. (2019). *Scikit-TDA: Topological data analysis for python*. https://doi.org/10.5281/zenodo.2533369

Tauzin, G., Lupo, U., Tunstall, L., Pérez, J. B., Caorsi, M., Medina-Mardones, A. M., Dassatti, A., & Hess, K. (2021). Giotto-tda:: A topological data analysis toolkit for machine learning and data exploration. *Journal of Machine Learning Research*, *22*(39), 1–6.

Van Veen, H. J., Saul, N., Eargle, D., & Mangham, S. W. (2019). Kepler mapper: A flexible python implementation of the mapper algorithm. *Journal of Open Source Software*, *4*(42), 1315.

Wadhwa, R. R., Williamson, D. F. K., Dhawan, A., & Scott, J. G. (2018). TDAstats: R pipeline for computing persistent homology in topological data analysis. *Journal of Open Source Software*, *3*(28), 860. https://doi.org/10.21105/joss.00860

Wadhwa, R., Piekenbrock, M., & Scott, J. (2020). *Ripserr: Calculate persistent homology with ripser-based engines*. https://doi.org/10.32614/CRAN.package.ripserr

Wickham, H. (2016). *ggplot2: Elegant graphics for data analysis*. Springer-Verlag New York. ISBN: 978-3-319-24277-4

You, K., & Yu, B. (2021). *TDAkit: Toolkit for topological data analysis*. https://doi.org/10.32614/cran.package.tdakit

DRAFT